টেস্টিং, মেইনটেন্যান্স, গুণমান ও মেট্রিক্স

চার স্তর · ডিবাগিং · CAPPP · SQA · গুণমান মডেল ও মেট্রিক্স: ১৩ পাতায় পরীক্ষার জন্য প্রস্তুত



এই নোটে যা আছে

- **টেস্টিংয়ের চার স্তর** — ইউনিট থেকে সিস্টেম পর্যন্ত
- **ভেরিফিকেশন বনাম ভ্যালিডেশন** — সবচেয়ে সাধারণ বিভ্রান্তি
- **ইন্টিগ্রেশনের চার কৌশল** — বিগ ব্যাং থেকে স্যান্ডউইচ
- **ডিবাগিংয়ের পাঁচ ধাপ** ও কৌশল
- **মেইনটেন্যান্সের চার প্রকার** — কারেক্টিভ থেকে প্রিভেন্টিভ
- **ম্যাককলের ১১টি গুণমান ফ্যাক্টর** তিনটি ক্যাটাগরিতে
- **মেট্রিক্সের তিন পরিবার** ও MTBF, MTTR

কীভাবে ব্যবহার করবেন: প্রথমে পাঁচটি স্তরের নাম ও প্রশ্ন মুখস্থ করুন, তারপর প্রতিটি সংজ্ঞার সাথে উদাহরণ জুড়ে দিন। পরীক্ষায় প্রায় সবসময় "সংজ্ঞা + উদাহরণ + ব্যতিক্রম" — এই তিনটি ধারাই আসে।

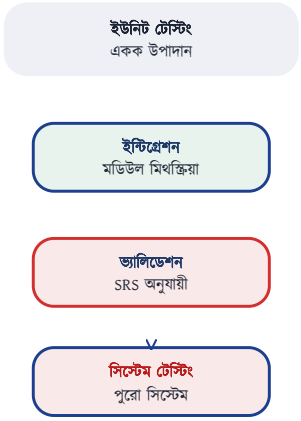
মনে রাখার সূত্র: **যাচাই** করা হয় "আমরা কি সঠিকভাবে তৈরি করছি?" আর **বৈধতা** যাচাই করা হয় "আমরা কি সঠিক জিনিস তৈরি করছি?"

১. ভেরিফিকেশন বনাম ভ্যালিডেশন

ভেরিফিকেশন — "আমরা কি সঠিকভাবে তৈরি করছি?" ডিজাইন স্পেসিফিকেশন অনুযায়ী আচরণ কি ঠিক? এটি **অভ্যন্তরীণ** কাজ, বিনা অনুমতিরেই চলে।

ভ্যালিডেশন — "আমরা কি সঠিক জিনিস তৈরি করছি?" গ্রাহকের প্রয়োজন পূরণ হচ্ছে কি? এর জন্য **SRS-এর সাথে তুলনা** করতে হয় এবং এতে **গ্রাহক** জড়িত।

> দুটো একসাথে **ভি ও ভি (V&V)**। ভ্যালিডেশন ছাড়া ভেরিফিকেশন অসম্পূর্ণ — সঠিকভাবে ভুল জিনিস বানানোও সম্ভব।



২. টেস্টিংয়ের চার স্তর, ক্রমে

1. **ইউনিট টেস্টিং** — পৃথক উপাদান বিচ্ছিন্নভাবে পরীক্ষা। সাধারণত JUnit দিয়ে স্বয়ংক্রিয়। সবচেয়ে সস্তা।
2. **ইন্টিগ্রেশন টেস্টিং** — মডিউলগুলি **কীভাবে একে অপরের সাথে যোগাযোগ করে** তা পরীক্ষা করে।
3. **ভ্যালিডেশন টেস্টিং** — SRS-এর ভিত্তিতে গ্রাহকের প্রয়োজন পূরণ হচ্ছে কি।
4. **সিস্টেম টেস্টিং** — সম্পূর্ণ সিস্টেম; কার্যকরী ও **অকার্যকরী** উভয়ই।

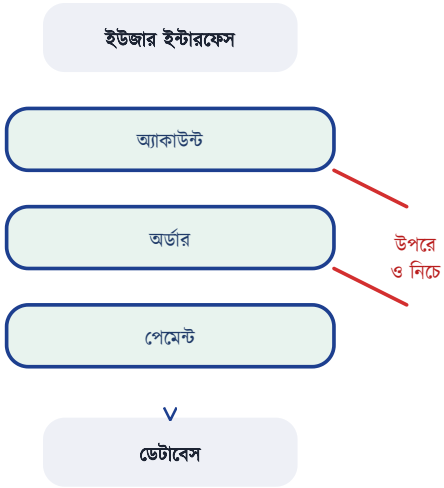
ভেরিফিকেশন **অভ্যন্তরীণ** — ডেভেলপার ও পরীক্ষক করে। ভ্যালিডেশন **বাহ্যিক** — এতে গ্রাহক বা প্রকৃত ব্যবহারকারী জড়িত।

> পাঁচটি স্তরের নাম এক বাক্যে: U-I-V-S-T — ইউনিট, ইন্টিগ্রেশন, ভ্যালিডেশন, সিস্টেম, টেস্টিং-অব-ইনস্টল।

৩. ইন্টিগ্রেশন টেস্টিং: চার কৌশল

কৌশল	ক্রম	সহায়ক
বিগ ব্যাং	উপর থেকে নিচে, সব একসাথে	ড্রাইভার
টপ-ডাউন	মূল থেকে নিচে ক্রমে	**স্টাব**
বটম-আপ	নিচ থেকে উপরে ক্রমে	**ড্রাইভার**
স্যান্ডউইচ	উপরে এক, নিচে এক — বার্তালী	দুটোই

স্টাব (Stub) হলো ক্ষণস্থায়ী নকল — কোনো মডিউলকে বদলে বসানো হয় যাতে উপরের মডিউল পরীক্ষা করা যায়, কিন্তু ভেতরে কাজ করে না। **ড্রাইভার** (Driver) এর উল্টো — নিচের মডিউলের সাময়িক কোড, যা পরীক্ষা কোডকে কল করে।



> পরীক্ষায় শুধু জোড়া মনে রাখুন: **স্টাব ↔ টপ-ডাউন**, **ড্রাইভার ↔ বটম-আপ**। টপ-ডাউনে উপরের মডিউল আগে আসে, তাই তার নিচেরটা নকল করা হয় = স্টাব। বটম-আপে নিচের মডিউল আগে আসে, তাই তার উপরেরটা কল করানো হয় = ড্রাইভার।

> বড় সিস্টেমে বাস্তবে **স্যান্ডউইচ** ব্যবহার হয়, কারণ এতে মিডলেয়ার পরীক্ষা দ্রুত হয় এবং বিভাগীয়ভাবে দল বিভক্ত করা যায়।

প্রশ্ন: টপ-ডাউন ইন্টিগ্রেশনে উপরের মডিউল পরীক্ষার জন্য নিচের মডিউলের যে স্থানীয় বিকল্প বসানো হয়, তার নাম কী? এবং সেটি কোন দিক থেকে আসে?

উত্তর: স্টাব — উপরের মডিউল থেকে নিচের দিকে (১-ক)

কৌশল বাছাইয়ের নিয়ম

পরিস্থিতি	বেছে নিন
উপরের মডিউল স্থির, নিচের বদলায়	টপ-ডাউন
নিচের মডিউল স্থির, উপরের বদলায়	বটম-আপ
দুই পাশই বদলায়, দ্রুত ফলাফল দরকার	স্যান্ডউইচ

> তিনটি কৌশলই আসলে একই সমস্যা সমাধান করে — **অসম্পূর্ণ মডিউলের বদলে সাময়িক কিছু বসানো**। পার্থক্য শুধু সেই সাময়িক জিনিসটি **উপরে** নাকি **নিচে** বসছে।

৪. ভ্যালিডেশন ও সিস্টেম টেস্টিং

ভ্যালিডেশন টেস্টিং — গ্রাহকের প্রয়োজনীয়তা পূরণ করে কিনা। SRS এর ভিত্তিতে হয়।

> **উদাহরণ:** "সিস্টেম ভুল পাসওয়ার্ড দিয়ে লগইন অনুমতি দেবে না" — এই শর্ত পূরণ হচ্ছে কি না পরীক্ষা করুন। ভুল পাসওয়ার্ডে অনুমতি দিলে ভ্যালিডেশন ব্যর্থ।

সিস্টেম টেস্টিং — ইন্টিগ্রেশনের পরে সম্পূর্ণ সিস্টেম পরীক্ষা করে। কার্যকরী ও অকার্যকরী উভয় অন্তর্ভুক্ত।

প্রকার	কী দেখে
লোড টেস্টিং	একসাথে অনেক ব্যবহারকারী
স্ট্রেস টেস্টিং	চরম কাজের চাপ
সিকিউরিটি টেস্টিং	SQL ইনজেকশন, XSS
কম্প্যাটিবিলিটি টেস্টিং	ব্রাউজার, ডিভাইস

> ভ্যালিডেশন বলে **"সিস্টেমটা ঠিক কাজ করছে"** আর সিস্টেম টেস্টিং বলে **"সিস্টেমটা ভাঙলেও টিকে থাকে"**। শেষটা ইউজারের জন্য সবচেয়ে জরুরি।

অযাচাইকৃত পথের গুরুত্ব

প্রতিটি টেস্ট কেবল একটি নির্দিষ্ট আচরণ যাচাই করে, পুরো সিস্টেম নয়। তাই অযাচাইকৃত পথ থেকে যায় এবং সেখান থেকেই বেশিরভাগ ত্রুটি বেরিয়ে আসে। তাই **পরীক্ষার পরিমাণ ও মান**—দুটোই গুরুত্বপূর্ণ।

> উদাহরণ: একটি ফর্মে ১০টি ঘর আছে, কিন্তু পরীক্ষা করা হয়েছে ৮টি বৈধ মানের জন্য। বাকি ২টি ঘরে যে ভুল ইনপুট ঢুকলে ক্র্যাশ হবে, তা পরীক্ষা ধরতে পারবে না। ★

লোড বনাম স্ট্রেস — পার্থক্যটা মনে রাখুন

	লোড	স্ট্রেস
উদ্দেশ্য	স্বাভাবিক ভার মাপা	সীমা ভাঙা
প্রশ্ন	কতজন সামলায়?	কোথায় ভাঙে?
ফলাফল	টাইম আউট	ত্রুটি, ক্র্যাশ

> উদাহরণ: একটি ওয়েবসাইট সাধারণত ১০০ জন ব্যবহারকারী সামলায় (লোড)। কিন্তু ৫০,০০০ জন একসাথে ঢুকলে কোথায় ভাঙে — সেটাই স্ট্রেস টেস্টিং। ★

প্রশ্ন: SQL ইনজেকশন কোন ধরনের টেস্টিংয়ে ধরা পড়ে?

উত্তর: সিকিউরিটি টেস্টিং (১-গ)

ভ্যালিডেশনের চার প্রচলিত কৌশল মনে রাখুন: **ক**োড চালিয়ে (স্ট্যাটিক), **দ**কুমেন্ট লিখিয়ে (পেপার), সরাসরি **ব্যবহার** করে, এবং **পরীক্ষা** করে।

৫. ডিবাগিং: কারণ খুঁজে বের করা ও সরানো

ডিবাগিং হলো ত্রুটি খোঁজা, বিশ্লেষণ এবং ঠিক করার প্রক্রিয়া। পাঁচটি ধাপ:

1. বাগ **পুনরুৎপাদন** করুন
2. অবস্থান **চিহ্নিত** করুন (লগ, ব্রেকপয়েন্ট)
3. **মূল কারণ বিশ্লেষণ** করুন
4. কোড **ঠিক** করুন
5. **পুনরায় পরীক্ষা** করুন

তিন নম্বর ধাপটাই আসল কাজ। বেশিরভাগ ছাত্র লক্ষণ দেখে সরাসরি কোড বদলে ফেলে, ফলে **মূল কারণ** বদলে **লক্ষণ** চলে যায়। পরীক্ষায় "ডিবাগিংয়ের পাঁচ ধাপ" জিজ্ঞেস করলেই এই ক্রম লিখতে হবে।

উদাহরণ (শূন্য দ্বারা ভাগ):

```
int divide(int a, int b) {
    if(b == 0)
        throw new IllegalArgumentException("b শূন্য হতে পারে না");
    return a / b;
}
```



ডিবাগিংয়ের কৌশল: প্রিন্ট ডিবাগিং (সবচেয়ে সহজ), **ডিবাগার টুল** (গাঢ়া কাজের জন্য), **বাইনারি সার্চ** (মাঝের অংশ মুছে দিয়ে অর্ধেক করে নেওয়া)।

ত্রুটির কয়েকটি সাধারণ কারণ

- **অফ-বাই-ওয়ান ত্রুটি** — শর্ত বা সীমানা ভুল লেখা
- **বিজলী ত্রুটি** — একই কোড একসাথে দুই জায়গায় বদলানো, এক জায়গা বাদ পড়ে
- **পড়া না পড়া ভুল** — ধরা পড়েনি এমন কোড
- **রাইস ত্রুটি** — দুই জায়গায় একই সময় লেখা হয়েছে

প্রথম ও দ্বিতীয়টি সবচেয়ে সাধারণ, আর তিন ও চতুর্থটি ব্যবহারকারীর চোখে ধরা পড়ে না বলেই কঠিন। **রাইস ত্রুটি** শনাক্ত করতে টেস্ট দরকার। ★

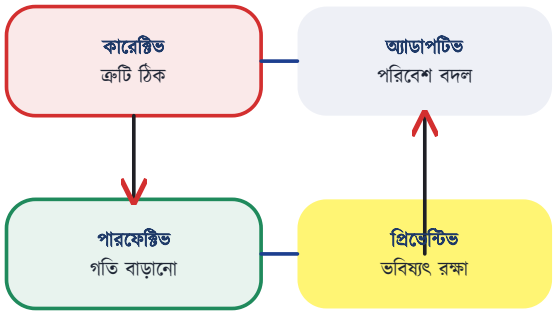
৬. সফটওয়্যার মেইনটেন্যান্স: কেন এটি আধিপত্য করে

গবেষণায় দেখা গেছে সফটওয়্যার জীবদ্দশায়কালের **৬০-৮০%** সময় মেইনটেন্যান্সে যায়, উন্নয়নে নয়। কারণ সফটওয়্যার অবিচ্ছিন্ন সেবা দেয় — গ্রাহকের চাহিদা বদলায়, পরিবেশ বদলায়, আইন বদলায়।

> একটি কারণ: সফটওয়্যারকে উৎপাদন শেষে "পাঠিয়ে দেওয়া" হয়, কিন্তু তা সারাজীবন **আপডেট ও পরিবর্তনের** জন্য তৈরি।

মেইনটেন্যান্সের চার প্রকার (CAPPP)

প্রকার	কখন হয়	উদাহরণ
কারেক্টিভ	ত্রুটি ঠিক করা	লগইন বাগ ঠিক করা
অ্যাডাপটিভ	পরিবেশ বদল	নতুন OS সমর্থন
পারফেক্টিভ	গতি বাড়ানো	SQL কোয়েরি দ্রুত করা
প্রিভেন্টিভ	ভবিষ্যতের জন্য	রিফ্যাক্টরিং



সব ধরনের কাজই মেইনটেন্যান্স

শুধু বাগ ফিক্স নয়

> মনে রাখার সূত্র: **কা-অ-পা-প্রি** — কারেক্টিভ বাগ, অ্যাডাপটিভ পরিবেশ, পারফেক্টিফ গতি, প্রিভেন্টিভ ভবিষ্যৎ। চারটিই আসলে "আমরা সঠিকভাবে বানিয়েছি কি না" যাচাই করে।

কারেক্টিভ মেইনটেন্যান্সের কয়েকটি রূপ

শব্দের অর্থ বদলানো — অন্যরা বোঝে না এমন শব্দ ব্যবহার। **ডেটা কাঠামো সংশোধন** — স্কিমা বা কোডের ভেতরের কাঠামো। **প্রান্তিক শর্ত সংশোধন** — দুই পরিস্থিতির মধ্যে সীমানা। **ডকুমেন্টেশন সংশোধন** — সহায়ক পাঠ বা নির্দেশনা।

> কারেক্টিভ মেইনটেন্যান্স প্রায় সবসময়ই চলে, তাই খরচের সবচেয়ে বড় অংশ এখানে। কিন্তু কারণ বোঝা যায় না বলে এটি সবচেয়ে **কম গণনীভুক্ত** কাজ। ★

প্রভাব বিশ্লেষণ কেন দরকার

প্রতিটি পরিবর্তন করার আগে জানতে হয় সেটি কোথায় কোথায় প্রভাব ফেলবে। ধরা যাক, লগইনের **পাসওয়ার্ড যাচাই** বদলাতে হলে — ডেটাবেস স্কিমা, ইন্টারফেস, টেস্ট কেস, ডকুমেন্টেশন, এমনকি গ্রাহকের অভ্যাসও প্রভাবিত হতে পারে।

> এই কারণেই ছোট পরিবর্তনেরও প্রভাব বিশ্লেষণ করা হয় — "ছোট বদল" বলে কিছু নেই। পরীক্ষায় "মেইনটেন্যান্সে প্রভাব বিশ্লেষণ কেন করা হয়?" — উত্তর: পরিবর্তনের পার্শ্বপ্রতিক্রিয়া ও ঝুঁকি কমাতে। ★

৭. মেইনটেন্যান্সের প্রক্রিয়া

প্রতিটি মেইনটেন্যান্স কাজ একই ছয় ধাপের ধারা অনুসরণ করে:

1. **পরিবর্তনের ধরন নির্ধারণ** — কোন প্রকারের মেইনটেন্যান্স (কারেক্টিভ নাকি অন্য কিছু)
2. **পরিবর্তন বিশ্লেষণ** — কী কী অবস্থান প্রভাবিত হবে
3. **ডিজাইন পরিকল্পনা** — কোথায় কী সম্পাদনা হবে
4. **কোড বাস্তবায়ন** — প্রকৃত সম্পাদনা
5. **পরীক্ষা ও যাচাই** — ভ্যালিডেশন সহ
6. **রিলিজ** — নতুন সংস্করণ প্রকাশ

> প্রথম ও তৃতীয় ধাপ সবচেয়ে গুরুত্বপূর্ণ কারণ এখানেই ভুল হলে পরে খরচ বাড়ে। পরীক্ষায় প্রায়ই "মেইনটেন্যান্স প্রক্রিয়ার প্রথম ধাপ কী?" — উত্তর: **পরিবর্তনের ধরন নির্ধারণ**। ★

> **ধাপ ২ ও ৫-এর মধ্যে পার্থক্য:** বিশ্লেষণ হলো "কী করতে হবে বুঝে নেওয়া", বাস্তবায়ন হলো "সেটা করা"। পরীক্ষা করে দেখা হয় কাজটা ঠিক হয়েছে কি না — এটাই যাচাই।

> **প্রক্রিয়াটি কেন এত সুনির্দিষ্ট**

মেইনটেন্যান্স ছাড়া কোনো সফটওয়্যার নির্ভরযোগ্যভাবে চলতে পারে না — বাস্তব সিস্টেমের সাথে এর একটি চুক্তি থাকে: পরিবেশ বদলালে সফটওয়্যারও বদলাতে হবে। তাই প্রতিটি ধাপ আলাদা করে নির্ধারণ করা হয়, যাতে কোনো ধাপ বাদ না পড়ে।

> একই কারণে মেইনটেন্যান্সের খরচ অভ্যন্তরীণ উন্নয়নের চেয়ে অনেক বেশি — কোডটি ইতিমধ্যে ব্যবহৃত অবস্থায় থাকে, অনেক ব্যবহারকারীর উপর নির্ভরশীল, এবং পরিবর্তনের পার্শ্বপ্রতিক্রিয়া অনেক বেশি। ★

প্রশ্ন: মেইনটেন্যান্স প্রক্রিয়ার দ্বিতীয় ধাপে কী করা হয়?

> উত্তর: পরিবর্তন বিশ্লেষণ — কোন কোন অবস্থান প্রভাবিত হবে তা নির্ধারণ (১-খ)

প্রশ্ন: ষষ্ঠ ধাপে কী হয়?

> উত্তর: রিলিজ — সংশোধিত সংস্করণ প্রকাশ (১-গ)

মেইনটেন্যান্সের খরচ সাধারণত অভ্যন্তরীণ উন্নয়নের চেয়ে অনেক বেশি হয়, কারণ কোডটি ইতিমধ্যে ব্যবহৃত অবস্থায় থাকে এবং পরিবর্তনের পার্শ্বপ্রতিক্রিয়া অনেক বেশি।

৮. সফটওয়্যার গুণমান নিশ্চয়তা (SQA)

SQA হলো সফটওয়্যারের গুণমান সম্পর্কিত সকল পরিকল্পিত ও কার্যক্রম — অর্থাৎ মান নিশ্চিত করার সব প্রক্রিয়ার সম্মিলন। এটি একক কোনো পরীক্ষা নয়, বরং **পুরো SDLC জুড়ে** ছড়িয়ে থাকা কার্যক্রম।

> SQA-র সাফল্য মাপা হয় **প্রক্রিয়া** দিয়ে, পণ্য দিয়ে নয়। অর্থাৎ "প্রতিটি ধাপে মান নিশ্চিত করার ব্যবস্থা আছে কি না" — এটিই SQA।

এসকিএএ-র প্রধান কাজ

- **মান নীতিমালা** তৈরি ও প্রয়োগ — মান কী, কীভাবে মাপব
- **রিভিউ** — কোড ও ডিজাইন পর্যালোচনা

> **CMMI স্তর — দ্রুত পরিচিতি**

স্তর	নাম	ভাবনা
১	প্রাথমিক	কাজ হয়, যত্ন নেই
২	পরিচালিত	পরিকল্পনা ও ট্র্যাক
৩	**সংজ্ঞায়িত**	**প্রক্রিয়া নির্ধারিত**
৪	পরিমাপিত	মাপা হয়
৫	অপ্টিমাইজড	উন্নতিমুখী

- **অডিট** — প্রক্রিয়া অনুসরণ করা হচ্ছে কি না দেখা
- **স্ট্যান্ডার্ড** — ISO 9000, CMMI-এর মতো মান নির্ধারণ
- পরীক্ষা দল গঠন ও **টেস্ট প্ল্যান** পর্যালোচনা
- ত্রুটির হিসাব রাখা ও **গুণমানের প্রতিবেদন**

যাচাইয়ের তিন স্তর

গুণমান নিশ্চিত করার কাজ তিন স্তরে চলে: **ডিজাইন পর্যালোচনা** (কাগজে), **কোড পর্যালোচনা** (লেখার পরে), এবং **পরীক্ষা** (চলার পরে)। প্রথম দুটি সাশ্রয়ী, তাই যত আগে করা যায় তত ভালো।

> পার্থক্যটা ধরুন: QC **(গুণমান নিয়ন্ত্রণ)** পণ্য পরীক্ষা করে — "এটা ঠিক আছে কি না?" আর QA **(গুণমান নিশ্চয়তা)** প্রক্রিয়া পরীক্ষা করে — "ঠিক হওয়ার ব্যবস্থা আছে কি না?"

> পরীক্ষায় "CMMI-তে সংজ্ঞায়িত প্রক্রিয়ার স্তর কোনটি?" — উত্তর: **স্তর ৩**। ★ এটি সবচেয়ে বেশি জিজ্ঞাসিত SQA প্রশ্নের একটি।

ISO 9001 কী

এটি একটি **আন্তর্জাতিক মান** যা সংস্থার মান ব্যবস্থাপনা ও প্রক্রিয়ার নির্ভরযোগ্যতা নিশ্চিত করে। সফটওয়্যার ক্ষেত্রে এটি সাধারণত মান ব্যবস্থাপনার সঙ্গে যুক্ত থাকে। CMMI যেখানে প্রক্রিয়াগুলো **পরিমাপযোগ্য** ও উন্নত করার লক্ষ্যে, ISO 9001 একই জিনিস আরেক ভঙ্গিতে বলে।

> দুটোর মধ্যে মূল পার্থক্য: CMMI **মডেলভিত্তিক** — প্রক্রিয়াগুলো কতটা পরিপক্ব। ISO **সনদভিত্তিক** — সংস্থা কি নির্ধারিত মান অনুসরণ করছে। ★

প্রশ্ন: SQA কোন একক পরীক্ষার নাম নয় কেন?

> উত্তর: এটি পুরো SDLC-তে ছড়িয়ে থাকা সব প্রক্রিয়ার সম্মিলন (১-ঘ)

৯. ম্যাককলের গুণমান ফ্যাক্টর

ম্যাককলের মডেলে ১১টি ফ্যাক্টর এবং তিনটি ক্যাটাগরি রয়েছে — পরীক্ষায় প্রায়ই এই ভাগটিই চাওয়া হয়।

ক্যাটাগরি	ফ্যাক্টরগুলো
অপারেশন	সঠিকতা, নির্ভরযোগ্যতা, কার্যকরতা, প্রতিরক্ষা, প্রাপ্যতা
রিভিশন	রক্ষণাবেক্ষণযোগ্যতা, পরীক্ষাযোগ্যতা, নমনীয়তা
ট্রানজিশন	পোর্টেবিলিটি, পুনর্ব্যবহারযোগ্যতা, ইন্টারঅপারেবিলিটি

> **একটি বাক্যে মনে রাখুন:** অপারেশন = এখন ঠিকভাবে কাজ করছে; রিভিশন = ভবিষ্যতে বদলানো সহজ; ট্রানজিশন = অন্য পরিবেশে নিয়ে যাওয়া যায়। ★

> পরীক্ষার শর্টকাট: **অ-রি-ট্রা** — অপারেশন, রিভিশন, ট্রানজিশন। ফ্যাক্টরগুলো আলাদা মুখস্থ করতে হলে "সি-রি-ফা-প্রা-ই-পো-পু-ই-ন" ধরনের সংক্ষিপ্ত আদ্যক্ষরে লেখা যায়।

অপারেশনের পাঁচটি — কী কী

- **সঠিকতা** — ফলাফল ঠিক কি না
- **নির্ভরযোগ্যতা** — নির্দিষ্ট সময়ে কাজ করে কি না
- **কার্যকরতা** — কম সময় ও মেমরিতে কাজ করে কি না
- **প্রতিরক্ষা** — অপব্যবহার থেকে রক্ষা করে কি না
- **প্রাপ্যতা** — সহজে ব্যবহারযোগ্য কি না

> পরীক্ষায় ভুল হয় এখানে: "কার্যকরতা" মানে কাজ দ্রুত ও কম সম্পদে করা, আর "নির্ভরযোগ্যতা" মানে নির্দিষ্ট সময়ে উপলব্ধ থাকা। দুটো আলাদা।

প্রশ্ন: কোন ফ্যাক্টরটি "ট্রানজিশন" ক্যাটাগরিতে পড়ে না?

> উত্তর: নির্ভরযোগ্যতা — এটি অপারেশনের ফ্যাক্টর (১-ক)

১০. সফটওয়্যার মেট্রিক্স: তিন পরিবার

পরিবার	কী মাপে
ফাংশন-ভিত্তিক	প্রোগ্রামের আকার — LOC, ফাংশন সংখ্যা
বহু-সাপেক্ষিক	তিন প্রকারের অন্তর্ভুক্তি
ধাপ-নির্দিষ্ট	নির্দিষ্ট ধাপের ফলাফল

বহু-সাপেক্ষিক (BANG) মেট্রিক্স চারটি গুণ করে — Bug প্রতি ইউনিট আউটপুট, Attributes প্রতি ইউনিট, Newlines প্রতি ইউনিট, Goal পালনের হার।

> পরীক্ষায় "BANG মেট্রিক্সে কয়টি উপাদান আছে?" — উত্তর: **৪টি** (B, A, N, G)। ★

১১. ত্রুটি, নির্ভরযোগ্যতা ও প্রাপ্যতার মেট্রিক্স

প্রাপ্যতা (Availability) হলো সিস্টেম যে সময়ের কত অংশ কাজ করে। এটি দুটি মেট্রিক্স দিয়ে হিসাব হয়:

> প্রাপ্যতা = MTBF ÷ (MTBF + MTTR)

মেট্রিক্স	পূর্ণ রূপ	কী মাপে
MTBF	গড় বিকল	দুর্ঘটনা হওয়ার গড় ব্যবধান
MTTR	গড় সময়	ঠিক করতে লাগা গড় সময়

> MTBF বাড়লে ও MTTR কমলে প্রাপ্যতা বাড়ে। পরীক্ষায় উদাহরণ হিসাব করতে দিতে পারে — সবসময় এই সূত্রটি লিখে নিন। ★

প্রশ্ন: MTBF-এর পূর্ণ রূপ কী?

উত্তর: Mean Time Between Failures (১-খ)

প্রশ্ন: MTTR কমলে প্রাপ্যতার উপর কী প্রভাব পড়ে?

উত্তর: প্রাপ্যতা বাড়ে (১-গ)

> **যা মনে রাখুন:** প্রাপ্যতা কখনো ১০০% হয় না, কারণ ঠিক করার সময়ও থাকে। তাই বাস্তবে ৯৫% বা তার বেশি প্রাপ্যতাই ভালো ধরা হয়।

ত্রুটি সংক্রান্ত মেট্রিক্স

- **ত্রুটির ঘনত্ব** = ত্রুটি সংখ্যা ÷ কোডের আকার
- **ত্রুটির হার** = ত্রুটি সংখ্যা ÷ মোট টেস্ট সংখ্যা × ১০০
- **MTTF** = প্রত্যাশিত আগে পর্যন্ত নির্বিঘ্ন চলার সময়

সরল উদাহরণ

ধরা যাক, সিস্টেমটি গড়ে প্রতি ২০ দিন পর পর একবার ব্যর্থ হয়, তাই **MTBF = ২০ দিন**। আর প্রতিবার মেরামতে ১ দিন লাগে, তাই **MTTR = ১ দিন**। তাহলে প্রাপ্যতা = ২০ ÷ (২০ + ১) = ২০ ÷ ২১ ≈ **৯৫.২%**।

> এই ধরনের হিসাব পরীক্ষায় সরাসরি আসে। শুধু সূত্রটি মনে রাখলেই যথেষ্ট — উত্তরে সূত্রটি লিখে দিলে নম্বর পাওয়া যায়। **≈৯৫%** বাস্তবসম্মত মান। ★

নির্ভরযোগ্যতা ও প্রাপ্যতার পার্থক্য

বিষয়	নির্ভরযোগ্যতা	প্রাপ্যতা
প্রশ্ন	ব্যর্থ হবে কি?	চালু থাকবে কি?
মাপে	MTBF	MTBF ও MTTR

উদাহরণ — সার্ভার মাসে একবার ব্যর্থ হয় (MTBF ≈ ৩০ দিন), মেরামতে ১ দিন (MTTR = ১), তবে প্রাপ্যতা = ৩০ ÷ ৩১ ≈ **৯৬.৮%**। ★

১২. ত্রুটি কেন আগে ধরা ভালো

একটি ত্রুটি যত দেরিতে ধরা পড়ে, তত বেশি খরচে তা ঠিক করতে হয়। কারণ প্রতিটি ধাপে সংশোধনের কাজ বাড়ে।

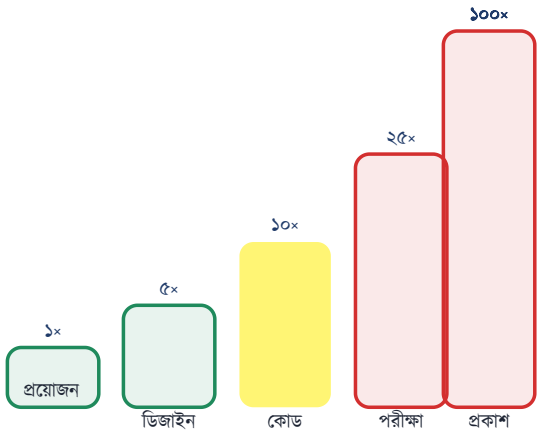
যেখানে ধরা পড়ল	আনুমানিক খরচের অনুপাত
প্রয়োজনীয়তা পর্যায়ে	× ১ — সবচেয়ে সস্তা
ডিজাইন পর্যায়ে	× ৫
কোড লেখার সময়	× ১০
পরীক্ষার সময়	× ২৫
প্রকাশের পরে	× ১০০ — সবচেয়ে ব্যয়বহুল

এই ক্রমটি মনে রাখতে সহজ উপায়: **১ → ৫ → ১০ → ২৫ → ১০০**। প্রতিবার ফ্যাক্টর প্রায় আড়াই গুণ বাড়ে। "১০০×" মনে রাখলেই পরীক্ষায় বেশিরভাগ প্রশ্নের উত্তর মিলে যাবে।

প্রকাশের পরে কেন এত বেশি খরচ

কারণ চারটি বিষয় একসাথে যুক্ত হয়: **পুরো কোড** আবার পরীক্ষা করতে হয়, **ডেটা মাইগ্রেট** করতে হয়, **ব্যবহারকারীর কাছ থেকে অভিযোগ** আসে, এবং সবচেয়ে বড় কথা — **সম্মান ও বিশ্বাস** নষ্ট হয়। শেষটি আর কোড দিয়ে কেনা যায় না।

> একটি প্রশ্ন সরাসরি আসতে পারে: "প্রয়োজনীয়তা পর্যায়ে ত্রুটি ধরলে খরচ কত বেশি?" — উত্তর: **×১**, অর্থাৎ সর্বনিম্ন। কারণ তখন কোডই লেখা হয়নি। ★



উপরের ধাপগুলো যত ডানে, খরচ তত বেশি। তাই **প্রয়োজনীয়তা পর্যায়েই ভুল ধরা সবচেয়ে সাশ্রয়ী**। ★

ত্রুটির জীবনচক্র

একটি ত্রুটি সাধারণত এই ধাপগুলো দিয়ে যায়: **শনাক্ত** → **রিপোর্ট** → **বিশ্লেষণ** → **সংশোধন** → **যাচাই** → **বন্ধ**।

ধাপ	যা ঘটে
শনাক্ত ও রিপোর্ট	ত্রুটি লিখিতভাবে জানানো হয়
বিশ্লেষণ	মূল কারণ ও প্রভাব ঠিক হয়
সংশোধন ও যাচাই	ঠিক করে পুনরায় পরীক্ষা

পরীক্ষায় "ত্রুটির জীবনচক্রে কোন ধাপে সবচেয়ে বেশি সময় লাগে?" — উত্তর: **বিশ্লেষণ ও সংশোধন**। কারণ ত্রুটি খুঁজে বের করা সহজ, কিন্তু তার মূল কারণ খুঁজে বের করাই কঠিন। ★

✓ অনুশীলনী MCQ (পরীক্ষার ধরন, ১০ নম্বর)

১. "আমরা কি সঠিক জিনিস তৈরি করছি?" — এই প্রশ্নটি কোন ধরনের যাচাই?

ক. ভেরিফিকেশন

খ. ইন্টিগ্রেশন টেস্টিং

গ. ভ্যালিডেশন

ঘ. সিস্টেম টেস্টিং

২. টপ-ডাউন ইন্টিগ্রেশনে নিচের মডিউলের স্থানে যা বসানো হয়?

ক. ড্রাইভার

খ. স্টাব

গ. মক

ঘ. প্রক্সি

৩. বটম-আপ কৌশলে উপরের মডিউলকে কল করানোর জন্য কী বসানো হয়?

ক. স্টাব

খ. ড্রাইভার

গ. ফেক

ঘ. সেন্টিনেল

৪. নতুন অপারেটিং সিস্টেমের সাপোর্ট যোগ করা কোন প্রকারের মেইনটেন্যান্স?

ক. কারেক্টিভ

খ. অ্যাডাপটিভ

গ. পারফেক্টিভ

ঘ. প্রিভেন্টিভ

৫. কোড দ্রুততর করার কাজকে কী বলে?

ক. কারেক্টিভ

খ. অ্যাডাপটিভ

গ. পারফেক্টিভ

ঘ. প্রিভেন্টিভ

৬. "সংজ্ঞায়িত প্রক্রিয়া" — এটি CMMI-এর কোন স্তর?

ক. ১

খ. ২

গ. ৩

ঘ. ৫

৭. কোন ফ্যাক্টরটি "রিভিশন" ক্যাটাগরিতে আছে?

ক. কার্যকরতা

খ. নমনীয়তা

গ. প্রাপ্যতা

ঘ. পোর্টেবিলিটি

৮. BANG মেট্রিক্সে কয়টি উপাদান আছে?

ক. ৩টি

খ. ৪টি

গ. ৫টি

ঘ. ৬টি

৯. প্রাপ্যতার সূত্র কোনটি?

ক. MTBF ÷ MTTR

খ. MTTR ÷ MTBF

গ. MTBF ÷ (MTBF + MTTR)

ঘ. MTTF + MTBF

১০. প্রকাশের পরে ত্রুটি ঠিক করার খরচ প্রায় কত বেড়ে যায়?

ক. ১০ গুণ

খ. ২৫ গুণ

গ. ১০০ গুণ

ঘ. ৫ গুণ

উত্তর: ১-গ · ২-খ · ৩-খ · ৪-খ · ৫-গ · ৬-গ · ৭-খ · ৮-খ · ৯-গ · ১০-গ

✓ সারসংক্ষেপ

- **টেস্টিং কৌশল:** ইউনিট → ইন্টিগ্রেশন → ভ্যালিডেশন → সিস্টেম ★
- **ডিবাগিং:** খোঁজা → বিশ্লেষণ → ঠিক করা → পুনরায় পরীক্ষা
- **মেইনটেন্যান্স প্রকার:** কারেক্টিভ, অ্যাডাপটিভ, পারফেক্টিভ, প্রিভেন্টিভ
- **SQA:** পুরো SDLC জুড়ে গুণমান নিশ্চিত করে
- **গুণমানের কারণ:** ম্যাককলের মডেল — অপারেশন, রিভিশন, ট্রানজিশন
- **খরচ প্রভাব:** আগে সনাক্তকরণ = সস্তা সমাধান
- **নির্ভরযোগ্যতা ও প্রাপ্যতা:** MTBF, MTTR মেট্রিক্স
- **মেট্রিক্স:** ফাংশন-ভিত্তিক, BANG, ধাপ-নির্দিষ্ট পরিমাপ

পার্শ্ববর্তী বিষয়: সাদা/কালো বাক্স টেস্টিং, চক্রিক জটিলতা, ভিত্তি পথ, সমতুল্য পরীক্ষা ও সীমানা মান বিশ্লেষণ রয়েছে **গ্র্যাম্প প্যাটার্ন ও ডিজাইন-লেভেল টেস্টিং**-তে; ওয়াটারফল, ভি-মডেল ও ইউজ কেস রয়েছে **ওয়াটারফল ও ইউএমএল**-এ।

পরীক্ষার আগে ✓

- ✓ চার স্তরের নাম ও প্রতিটির প্রশ্ন বলতে পারুন
- ✓ স্টাব ও টপ-ডাউন, ড্রাইভার ও বটম-আপ — দ্বিমুখী সম্পর্ক ভুলনা দিয়ে বলুন
- ✓ চার প্রকার মেইনটেন্যান্স উদাহরণ দিয়ে ব্যাখ্যা করুন
- ✓ ম্যাককলের ১১টি ফ্যাক্টর সঠিক ক্যাটাগরিতে লিখতে পারুন
- ✓ MTBF ও MTTR থেকে প্রাপ্যতা হিসাব করুন
- ✓ ত্রুটির খরচের ক্রম ১ → ৫ → ১০ → ২৫ → ১০০ বলুন